

1. Introduzione

L'obiettivo dell'assignment è progettare e implementare una libreria concorrente per l'analisi ricorsiva del filesystem, in grado di:

- visitare una directory e tutte le sue sotto-directory
- classificare i file in fasce di dimensione (bands)
- aggregare i risultati in un report finale

Il problema è particolarmente interessante dal punto di vista della concorrenza perché combina:

- **operazioni I/O-bound** (lettura directory, lettura metadati file)
- **ricorsione potenzialmente profonda**
- **parallelizzazione naturale** (ogni directory è indipendente)
- **aggregazione di risultati parziali**

Sono state sviluppate tre versioni con approcci che se pur tutti concorrenti, molto diversi fra loro, ognuno dei quali presenta vantaggi e svantaggi che verranno esaminati in seguito.

2. Analisi del problema dal punto di vista concorrente

La scansione del filesystem presenta tre caratteristiche fondamentali:

Il problema è fortemente I/O-bound => L'operazione dominante è `listFiles()` / `readDir()`, che richiede accesso al disco. Il tempo di CPU è minimo, la latenza del disco è il fattore limitante. La natura I/O-bound del problema è ancora più evidente su macchine che sfruttano hard disk meccanici, dove la latenza di accesso al disco è significativamente più elevata rispetto agli SSD. In questi scenari, la concorrenza risulta ancora più rilevante, poiché il parallelismo permette di "mascherare" parzialmente la latenza del disco e ridurre sensibilmente il tempo totale di scansione.

La ricorsione è naturalmente parallelizzabile => Ogni directory può essere scansionata indipendentemente dalle altre, questo oltre a ridurre la complessità generale permette di lanciare task in modo parallelo, sfruttare thread multipli il che si traduce dal punto di vista dell'utente nel ridurre drasticamente il tempo totale di attesa.

È necessario aggregare risultati parziali => Ogni directory produce un report locale che deve essere combinato con quelli delle sotto-directory. Poiché la scansione del filesystem è parallela, ogni sotto-directory produce un report parziale che deve essere combinato con gli altri. Nelle versioni Rx e Virtual Threads, questi report vengono generati da thread diversi, quindi è necessario che siano immutabili: nessun thread deve modificare lo stesso oggetto, altrimenti si introdurrebbero race condition. L'immutabilità garantisce che ogni merge sia

sicuro e privo di interferenze. Nella versione Vert.x, invece, il report è mutabile, ma viene aggiornato esclusivamente dall'event-loop, che è single-thread, il che elimina alla radice ogni problema di concorrenza, permettendo l'uso di una struttura dati mutabile senza rischi.

3. Strategie adottate

3.1 Versione Event-Loop (Vert.x)

La versione Vert.x è costruita attorno al modello event-driven, basato su:

- un event-loop (single thread)
- un pool di background thread (worker) per operazioni bloccanti
- callback e future non bloccanti

Struttura:

- FSStatLib crea un verticle e restituisce subito una Future<FSReport>.
- FSScanVerticle esegue la scansione ricorsiva.
- fs.readDir() e fs.props() sono chiamate asincrone delegate ai worker thread.
- L'event-loop aggiorna un FSReport mutabile, sicuro perché aggiornato da un solo thread.

Ricorsione:

La scansione ricorsiva in Vert.x è strutturata attorno al metodo scanDirectory(), che viene invocato una volta per ogni directory incontrata nell'albero. Ogni chiamata crea una *promise locale* che rappresenta il completamento della scansione del sottoalbero radicato in quella directory.

La ricorsione procede così:

1. **Lettura asincrona della directory:** La chiamata a fs.readDir(dir) è completamente non bloccante, Vert.x delega l'operazione ai worker e l'event-loop continua a processare altri eventi.
2. **Per ogni entry della directory**
 - Se l'entry è un file, l'event-loop aggiorna immediatamente il report locale tramite report.addFile(size).
 - Se l'entry è una directory, viene avviata una nuova chiamata ricorsiva a scanDirectory(), che restituisce una Future<FSReport> relativa al sottoalbero.

3. **Gestione delle future**

Il codice mantiene due liste:

- propsFutures: future che rappresentano il completamento delle operazioni props() su ogni entry (file o directory).
- subDirFutures: future che rappresentano il completamento delle chiamate ricorsive sulle sotto-directory.

Questo doppio livello è necessario perché:

- props() deve completarsi per *tutte* le entry,
- ma solo alcune entry generano future aggiuntive (quelle che sono directory).

4. Sincronizzazione finale

Quando tutte le future in propsFutures sono completate, viene chiamato “Future.all(propsFutures)”. A questo punto tutte le entry sono state processate, tutte le sotto-directory hanno completato la loro scansione e i vari report di esse sono disponibili in “subDirFutures”. L’event loop può eseguire il merge e completare la promise con il report aggregato che era stata creata dalla libreria e passata al verticle.

Estensione interattiva:

L’estensione interattiva è stata realizzata sulla base del modello con event-loop, in particolare le differenze principali rispetto alla versione standard sono le seguenti:

- **Report parziale:** (FSReportPartial) è mutabile, ma viene aggiornato *solo* dall’event-loop, questo elimina completamente la necessità di sincronizzazione. E’ necessario introdurre il concetto di report parziale visto che, nel caso in cui venga premuto il pulsante “stop” bisogna avere un risultato pronto da restituire.
- **EventBus:** Ogni UPDATE_EVERY file, il verticle pubblica uno snapshot del report sul topic fs.update. La GUI Swing è in ascolto e aggiorna la text area.
- **Stop cooperativo:** Un AtomicBoolean viene controllato all’inizio di ogni chiamata ricorsiva e prima di ogni props(). Se il flag è attivo, la scansione si interrompe immediatamente e la promise viene completata con il report parziale.

3.2 Versione Reactive (RxJava)

La versione basata su **RxJava** affronta la scansione del filesystem adottando un modello completamente diverso rispetto al precedente. L’idea centrale è trasformare l’intero albero delle directory in un *Flowable<File>* che emette tutti i file presenti nella directory radice e nelle sue sotto-directory.

Pipeline reattiva:

La costruzione del flusso avviene tramite il metodo scanDir(), che utilizza Flowable.defer() per garantire che la scansione non inizi immediatamente, ma solo quando qualcuno si iscrive al

flusso (flusso “cold”). Questo è un aspetto importante: la pipeline è definita in anticipo, ma l’esecuzione parte solo al momento della `subscribe()`.

Per quanto riguarda la lettura delle directory viene sfruttato il metodo “`subscribeOn`” sul quale viene specificato l’uso di uno scheduler specifico (`Schedulers.io()`). In questo modo ogni directory viene letta su uno scheduler I/O dedicato, permettendo di parallelizzare naturalmente la visita dell’albero. Tramite l’approccio RxJava ci possiamo permettere di non creare thread manualmente, la gestione della concorrenza viene delegata allo scheduler, che decide quando e come eseguire le operazioni, il tutto permette quindi di avere una visione più “ad alto livello” del problema.

Ricorsione tramite `flatMap`:

La ricorsione non è implementata tramite chiamate dirette, ma tramite la composizione dei flussi:

- se l’entry è un file => viene emesso come singolo elemento del flusso.
- se l’entry è una directory => `scanDir()` viene richiamato ricorsivamente e il flusso risultante viene “appiattito” nel flusso principale tramite `flatMap`.

Aggregazione funzionale tramite `reduce`:

Una volta ottenuto un flusso di file, la pipeline applica una trasformazione da file a `file.length()`, infine viene tutto aggregato in un unico report immutabile. Il metodo `withFile()` infatti crea un nuovo report immutabile ad ogni aggiornamento, poiché la pipeline può essere eseguita in parallelo, l’immutabilità garantisce che non ci siano race condition: ogni step produce un nuovo oggetto, e il `reduce` combina questi oggetti sfruttando uno stile funzionale.

Sottoscrizione e completamento:

La pipeline restituisce un `Flowable<FSReportRx>` che emette un solo elemento: il report finale. È importante sottolineare che il main utilizza `blockingSubscribe()` per attendere il completamento della pipeline, ma la libreria in sé rimane completamente non bloccante.

3.3 Versione Virtual Threads

La versione basata sui Virtual Threads sfrutta il modello di concorrenza che permette di creare un numero molto elevato di thread a costo estremamente ridotto. Questo consente di adottare un approccio “thread-per-task” anche in scenari ricorsivi e I/O-bound come la scansione del filesystem, mantenendo un codice imperativo semplice e leggibile, l’API per l’uso dei Virtual Threads rimane infatti molto simile a quella dell’uso di `careers threads`.

La struttura della libreria è centrata sul metodo `getFSReport()`, che crea un executor tramite `Executors.newVirtualThreadPerTaskExecutor()`. Questo executor non riutilizza i thread, bensì

ogni chiamata a `submit()` crea un nuovo virtual thread dedicato. L'executor rimane attivo per tutta la durata della ricorsione e viene chiuso automaticamente al termine.

La logica principale risiede nel metodo ricorsivo `scanDir()`. Per ogni directory, il metodo:

1. **Legge le entry:** Se la directory non è accessibile, viene restituito un report vuoto.
2. **Processa i file localmente:** I file vengono gestiti immediatamente nel thread corrente, aggiornando il report immutabile tramite `withFile()`. Questo approccio mantiene il codice semplice e privo di effetti collaterali.
3. **Lancia la ricorsione parallela sulle sotto-directory:** Per ogni sotto-directory, viene creato un nuovo virtual thread tramite "`executor.submit(() -> scanDir(entry, ...))`", la chiamata restituisce una `Future<FSReportVT>`, che rappresenta il risultato della scansione del sottoalbero, queste future vengono raccolte in una lista. Un aspetto importante da chiarire riguarda la differenza tra le Future utilizzate nella versione con Virtual Threads e quelle utilizzate nella versione Vert.x. Sebbene condividano il nome, si tratta di astrazioni profondamente diverse, progettate per modelli di concorrenza opposti. In questa versione vengono utilizzate le Future di `java.util.concurrent`, questa Future è **bloccante**: la chiamata a `f.get()` sospende il thread corrente finché il risultato non è disponibile. Nel contesto dei Virtual Threads questo non rappresenta un problema, perché i Virtual Thread sono estremamente leggeri e possono essere sospesi e ripresi senza costi significativi. Nella versione con Vert.x vengono utilizzate le Future di `io.vertx.core` che è una Future completamente **non bloccante**, progettata per integrarsi con l'event-loop. Il risultato viene gestito tramite callback (`onSuccess`, `onFailure`) e propagato attraverso una catena di operazioni asincrone. L'event-loop non può mai essere bloccato, quindi l'intero modello si basa sulla composizione di callback e promise.
4. **Attende i risultati delle sotto-directory:** Dopo aver processato tutti i file locali, viene eseguita la fase di merge.

L'uso di un report immutabile (`FSReportVT`) è coerente con il modello: ogni virtual thread produce un proprio report locale, e il merge avviene solo dopo che i thread hanno terminato. Non vi è mai accesso concorrente allo stesso oggetto, e quindi non è necessaria alcuna forma di sincronizzazione. Nel complesso, la versione con Virtual Threads permette di esprimere la ricorsione in modo naturale, quasi identico alla versione sequenziale, ma con un parallelismo molto più efficace. Il codice rimane lineare e leggibile: non ci sono callback, non ci sono future non bloccanti, non ci sono scheduler o operatori reattivi. La concorrenza è ottenuta semplicemente lanciando un thread per ogni sotto-directory.

4. Conclusioni

L'analisi delle tre implementazioni mostra come modelli di concorrenza profondamente diversi possano affrontare lo stesso problema con approcci e risultati molto differenti.

La versione **Vert.x**, basata su un event-loop e su operazioni asincrone delegate ai worker thread, evidenzia i punti di forza e i limiti del paradigma event-driven. Da un lato, il modello non bloccante permette di mantenere reattività e di integrare facilmente funzionalità interattive come aggiornamenti dinamici e stop cooperativo. Dall'altro, la necessità di comporre callback promise e future non bloccanti rende il codice più complesso da seguire, e l'uso di I/O bloccante come `readDir()` e `props()` introduce un overhead significativo, rendendo questa soluzione meno efficiente rispetto alle altre due.

La versione **RxJava** adotta un paradigma completamente diverso, modellando la scansione come un flusso di dati trasformato e aggregato tramite operatori funzionali. Questo approccio permette di esprimere la ricorsione e la parallelizzazione in modo dichiarativo, delegando la gestione dei thread allo scheduler I/O. La pipeline risulta compatta e modulare, e l'uso di un report immutabile elimina alla radice i problemi di concorrenza. Tuttavia, l'overhead introdotto dagli scheduler e la natura I/O-bound del problema rendono questa soluzione meno performante rispetto a un modello più diretto come quello dei Virtual Threads.

La versione basata sui **Virtual Threads** rappresenta l'approccio più naturale per questo tipo di problema. Il modello thread-per-task, reso possibile dalla leggerezza dei virtual thread, permette di esprimere la ricorsione in modo quasi identico alla versione sequenziale, ma con un parallelismo molto più efficace. L'uso di future bloccanti non introduce penalizzazioni, poiché i virtual thread possono essere sospesi e ripresi senza costi significativi. Il risultato è un codice lineare, leggibile e privo di complessità strutturale, che sfrutta appieno il parallelismo disponibile senza richiedere callback, scheduler o strutture reattive.

In sintesi, le tre versioni mostrano come la scelta del modello concorrente debba essere guidata dalla natura del problema. Per scenari fortemente I/O-bound e ricorsivi, come la scansione del filesystem, i Virtual Threads offrono la soluzione più semplice ed efficiente. RxJava rappresenta un'alternativa elegante e funzionale, mentre Vert.x si distingue soprattutto per la capacità di integrare funzionalità interattive, pur non essendo il modello più adatto dal punto di vista prestazionale.